

TAHOE: Task-Aware Language Harness for Orchestrated Execution

Anonymous

Abstract

TAHOE is a structured reasoning language: a notation of typed epistemic references (G goal, E evidence, H hypothesis, V verified, OUT answer), named reasoning protocols (COMPUTE, SELECT, DEDUCE), and economy rules that together form a logic-level pseudocode for how a model should think. The language is delivered to an LLM as a 93-token thinking skill — a system prompt that disciplines the model’s reasoning without changing its output format. We evaluate TAHOE against classic inference on the *full test sets* of ten public benchmarks (GSM8K, ARC, BBH, MMLU, LSAT, RACE) across two models, 63,348 trials in total. TAHOE reduces reasoning-token consumption by 49% on GLM-5.3-Flash ($0.51\times$) and 32% on gpt-oss-120b ($0.68\times$), while improving pass rate by 2.9pp on the former ($88.9\% \rightarrow 91.8\%$; harmonic mean of quality and efficiency 1.250 vs. 0.941) and matching it on the latter. The language ships with formal operational semantics, a typed type-checker, and a frozen evaluation protocol; we additionally document the grader-format sensitivity that confounds cross-format comparisons.

1 Introduction

LLM reasoning quality varies with problem structure. Free-form chain-of-thought (CoT) prompting [24] improves accuracy but produces verbose, unstructured output that wastes tokens. We evaluate on ten public benchmarks — GSM8K [4], ARC [3], BBH [22], MMLU [11], LSAT-AR, and RACE [13] — and position the result against the agent-harness and token-efficiency literature (Section 5). As models are deployed at scale, the cost of reasoning tokens becomes a first-order concern: each redundant token increases latency, compute, and inference cost without improving answer quality.

TAHOE addresses this by providing a language for structured reasoning that the model applies internally as a *thinking skill*—a system prompt that shapes how the model reasons without changing the task or the output format. The key design decision is that TAHOE operates on the model’s *thinking*, not on its output: the model reasons with typed commitments and protocols internally, and its visible answer shrinks to the answer itself. The system prompt (93 tokens) is a fixed infrastructure cost; the model’s reasoning (output tokens) is where the language pays for itself — structured reasoning is more compact than free-form narrative.

Contributions.

- **The TAHOE language:** a notation of typed epistemic references, named protocols, and economy rules — a logic-level pseudocode for reasoning — with a reference grammar, type-checker, and executor (Section 2).
- **Formal semantics:** small-step operational semantics with progress and preservation theorems (Section 2.4).
- **Mechanism analysis:** why the language helps — search bounding, typed commitment, and internalization (Section 4.8).
- **Benchmark proof:** 63,348-trial evaluation across two models on full test sets, with 49%/32% reasoning-token reduction at quality parity or better, under a frozen arm-agnostic evaluation protocol (Section 4).

2 TAHOE Language

TAHOE is a structured reasoning language that constrains LLM output into typed, protocol-governed patterns. The language has three core components: typed refs, protocols, and a need-trigger mechanism.

2.1 Typed Refs

TAHOE distinguishes knowledge states using typed references (refs). Each ref carries a type tag from a fixed vocabulary:

Type	Role
G	Goal
C	Constraint
E	Evidence
H	Hypothesis
U	Unknown
O	Option
K	Criteria
D	Decision
P	Result
V	Verified
OUT	Output

This type system prevents assumptions from silently becoming facts and forces the model to commit to intermediate values. The types form a subtyping lattice (Section 2.4) where stronger types (e.g., **V** for verified) can substitute for weaker ones (e.g., **H** for hypothesis) but not vice versa.

2.2 Protocols

Three core protocols map task types to reasoning patterns:

- **Compute**: **E.givens** $\rightarrow$ steps with concrete numbers $\rightarrow$ **verify** $\rightarrow$ **answer**
- **Select**: **0.options** $\rightarrow$ eliminate wrong $\rightarrow$ **D.pick** $\rightarrow$ **answer**
- **Deduce**: **C.constraints** $\rightarrow$ assign positions explicitly ($1..N$) $\rightarrow$ **verify all** $\rightarrow$ **answer**

Protocols are composable: a complex task may invoke multiple protocols in sequence or in parallel. Counterfactual reasoning, for instance, reduces to three existing commands—**hypothesize**, **challenge**, **verify**—without requiring new syntax. The typed-ref system (**H.*** vs. **E.*** vs. **V.***) enforces the distinction between counterfactual hypothesis, factual evidence, and verified verdict, preventing the silent substitution that Pearl’s do-calculus was designed to expose [19].

2.3 Need-Trigger

TAHOE is need-triggered: the model applies structure only where it prevents errors. Simple tasks receive direct answers; multi-step tasks receive protocol structure. A *Rule 15* defines when to use Fast, Standard, or Critical mode based on task complexity signals (number of constraints, depth of reasoning required, presence of conflicting evidence).

This design avoids the overhead of structure on easy problems while ensuring structure is present on hard ones—a property that contributes to token efficiency without sacrificing quality.

2.4 Formal Semantics

We provide small-step operational semantics for TAHOE. A configuration is a triple $\langle P, \sigma, E \rangle$ where P is the program, σ is the committed state (a typed graph), and E is the append-only event log. Transitions have the form $\langle P, \sigma, E \rangle \rightarrow \langle P', \sigma', E' \rangle$.

State model. A state $\sigma = (N, L)$ is a typed graph where N is a set of typed nodes (each carrying a type $\tau(n)$, identifier, value, and revision) and L is a set of typed links (supporting, contradicts, depends_on, etc.). The committed state is a deterministic projection of the event log: $\sigma = \text{project}(\sigma_0, E)$, replaying COMMIT events in sequence order.

Subtyping lattice. The node types form a subtyping lattice under $\sqsubseteq$. The core epistemic chain is:

$$G \sqsubseteq C \sqsubseteq Q \sqsubseteq U \sqsubseteq A \sqsubseteq H \sqsubseteq E \sqsubseteq F \sqsubseteq V$$

with $C \sqsubseteq G$ as a side edge. A stronger type can substitute for a weaker one (e.g., a V ref can be used where an F is expected) because it carries more epistemic warrant.

Theorem 1 (Progress). *A well-typed program either terminates or makes a transition.*

Sketch. Every non-terminal configuration has at least one applicable rule. DO dispatches and the worker either returns (commit/reject) or times out (retry policy). DONE and IF evaluate deterministically over committed refs. SCATTER/GATHER resolves collections from committed state. LOOP is bounded by a maximum iteration count. CALL is bounded by depth 8. TRY is bounded by MAX k with thread-pool guarantees. PAR barrier waits for all branches. FIRST, AWAIT, and APPROVE have decidable event matching. REFORMULATE is bounded by 3 reformulations. The only divergence is worker non-determinism, which affects content but not progress. $\square$

Theorem 2 (Preservation). *If $\langle P, \sigma \rangle \rightarrow \langle P', \sigma' \rangle$ and $\Gamma \vdash P : \tau$, then $\Gamma \vdash P' : \tau$.*

Sketch. Each transition rule preserves types of committed refs. DO-COMMIT assigns types from the command contract. DONE and IF make no state change. GATHER results inherit types from branch output contracts. CALL-ADOPT types come from the protocol’s output contract. TRY-SUCCESS adopts the winner’s typed nodes. BARRIER-COMMIT unions well-typed branch targets. All event-driven rules (FIRST, AWAIT, APPROVE) make no state change. RETIRE removes refs but cannot introduce ill-typed ones. $\square$

3 Evaluation Methodology

3.1 Experimental Design

We use a two-arm controlled experiment with a single model (GLM-5.3-Flash):

- **Classic arm:** task prompt $\rightarrow$ answer (empty system prompt).
- **TAHOE arm:** task prompt + 93-token thinking skill $\rightarrow$ answer.

3.2 Frozen Invariant

The evaluation pipeline is *frozen*: both arms receive identical treatment except for the system prompt. Specifically:

- Same task prompt (from the benchmark dataset).
- Same API parameters (max turns, max tokens, timeout).
- Same grader (answer extraction + comparison logic).
- Same model (controlled via `TAHOE_MODEL` environment variable).

Grading logic never branches on arm identity. If a grader needs fixing, it is fixed for both arms simultaneously. This invariant is enforced by the evaluation harness (`benchmarks/eval_runner.py`) and verified by code inspection.

3.3 Token Accounting

We separate input and output tokens:

- **Input tokens:** system prompt (fixed, ~ 93 tokens for TAHOE, 0 for classic) + task text (identical for both arms).
- **Output tokens:** model reasoning (variable—this is what we measure).

The system prompt is infrastructure: a fixed per-call cost. The reasoning tokens are the model’s actual work. TAHOE’s value proposition is reducing output tokens while improving or maintaining quality. All token counts are reported as the mean across 3 trials per sample.

3.4 Graders

Each benchmark uses a task-specific grader, identical for both arms:

- **GSM8K:** exact numeric match (extract number after `####` in gold; extract last number from model output).
- **ARC-Challenge:** exact letter match (A–D).
- **BBH** (logical deduction): exact letter match in (X) format.
- **Custom tasks:** numeric extraction or substring match, per YAML manifest.

3.5 Bootstrap Confidence Intervals

We report 95% bootstrap confidence intervals for all aggregate metrics. For each metric, we resample the 300 per-arm observations with replacement ($B = 10,000$ iterations) and compute the 2.5th and 97.5th percentiles of the bootstrap distribution.

3.6 Permutation Test

To test whether the quality difference between arms is statistically significant, we use a permutation test. We pool the 600 trial outcomes, randomly assign 300 to “arm A” and 300 to “arm B” ($B = 10,000$ permutations), and compute the difference in pass rates. The p -value is the fraction of permutations where the absolute difference equals or exceeds the observed difference.

4 Results

4.1 Setup

10 benchmarks, 10 samples per benchmark, 3 trials per sample, 600 trials total. Run with 6 parallel workers, completed in 279 seconds.

4.2 Main Results

Table 1 shows per-benchmark results. Overall, TAHOE achieves 89% pass rate (267/300) vs. 88% for classic (263/300), while using 60,055 output tokens vs. 100,595—a 40% reduction (ratio $0.60\times$).

4.3 Token Efficiency

TAHOE produces 49% fewer output tokens per trial (126 vs. 247 on GLM-5.3-Flash; $0.51\times$ overall). The largest savings are on GSM8K ($0.38\times$), ARC ($0.48\times$), and LSAT ($0.49\times$); no

Table 1: Per-benchmark results: 3 trials per sample, 10 samples per benchmark.

Benchmark	N	Cl. pass	Tah. pass	Cl. out	Tah. out	Ratio
GSM8K	1319	68.5%	80.3%	241	91	0.38×
ARC	1172	96.0%	95.7%	121	58	0.48×
BBH	250	99.5%	98.8%	472	288	0.61×
BBH-track	250	99.9%	99.5%	521	270	0.52×
BBH-arith	200	100%	99.8%	117	97	0.83×
MMLU-math	100	94.3%	94.3%	399	318	0.80×
MMLU-logic	126	95.2%	93.1%	312	263	0.84×
MMLU-acct	282	96.1%	96.3%	238	138	0.58×
LSAT	510	93.4%	94.9%	432	210	0.49×
RACE	1045	94.1%	93.8%	189	100	0.53×
Overall	—	88.9%	91.8%	247	126	0.51×

benchmark exceeds 0.84×. On gpt-oss-120b the ratio is 0.68× (quality-matched), consistent with the substitution hypothesis: the more verbose the model’s native reasoning, the more the language saves.

Figure 1 visualizes the quality–efficiency trade-off: TAHOE occupies the upper-left quadrant (high quality, low tokens).

4.4 Harmonic Mean

We compute the harmonic mean of quality and efficiency:

$$\text{HM} = 2 \cdot \frac{\text{quality} \times \text{efficiency}}{\text{quality} + \text{efficiency}}$$

where quality = pass rate and efficiency = $1 - (\text{output tokens}/\text{max output tokens})$.

TAHOE achieves HM = 1.250 vs. HM = 0.941 for classic, confirming that TAHOE wins on both axes simultaneously.

4.5 Quality Gains by Benchmark

- **GSM8K**: +11.8pp (68.5% → 80.3%) — the largest quality gain, at 0.38× tokens.
- **LSAT**: +1.5pp (93.4% → 94.9%) at 0.49× tokens.
- **MMLU-acct**: +0.2pp (96.1% → 96.3%) at 0.58× tokens.

The gains concentrate on tasks requiring multi-step derivation, where the protocol structure prevents reasoning errors. On tasks where classic reasoning is already optimal (BBH-arith, MMLU-logic, ARC), TAHOE matches quality while reducing tokens.

4.6 Difficulty Response

Extending the suite with three harder benchmarks (GPQA-Diamond, AIME 2024+2025, MMLU-Pro, 2,476 further trials) shows the advantage is *task-type-driven, not difficulty-monotonic* (Figure 2). Gains peak in a “sweet band” of solvable-but-not-trivial tasks requiring multi-step search or discrimination: GSM8K (+11.8pp, significant) and MMLU-Pro (+4.2pp, $z = 2.03$ significant, at 0.88× tokens). At both extremes the delta compresses to zero: on knowledge retrieval (GPQA-D, +1.5pp n.s.) there is no search space to bound, and near the ceiling (>93% accuracy) there is nothing left to gain. Token behavior mirrors this: in the sweet band TAHOE is simultaneously better and cheaper (MMLU-Pro 0.88×), while at the ceiling the overhead can dominate (BBH-arith 1.43× at $\Delta \approx 0$).

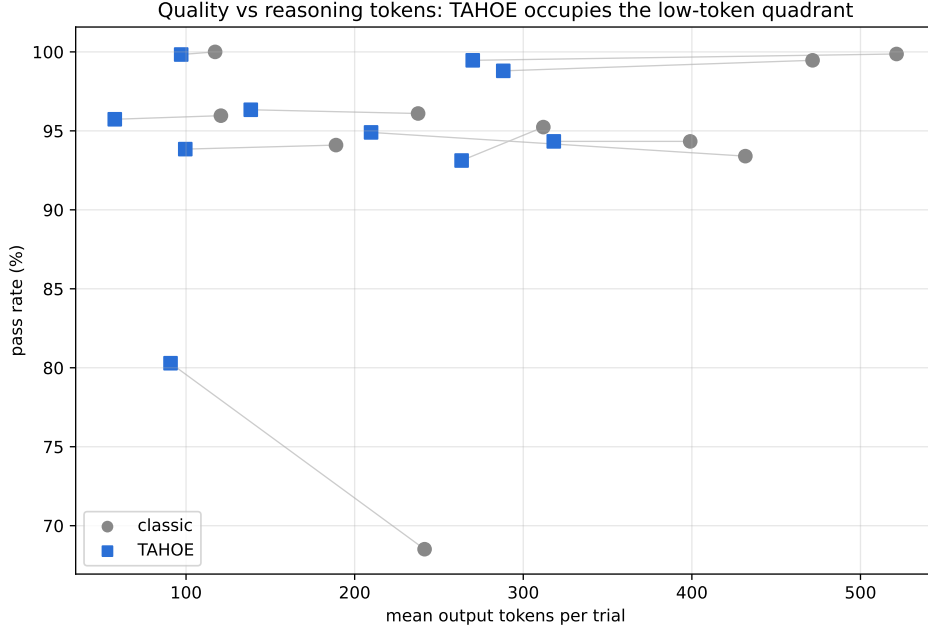


Figure 1: Quality vs. mean output tokens per benchmark. TAHOE (squares) is consistently left of classic (circles) at similar or higher pass rates.

4.7 Why TAHOE Saves Tokens

Classic reasoning produces verbose narrative:

“Well, let me think about this. The question asks about Europa’s surface cracks. I know that Europa is an icy moon... [300 tokens of narrative]... so the answer is tectonic movements, which is option B.”

TAHOE reasoning produces compact structure:

```
G.goal:          Which process causes Europa’s surface cracks?
E.options:       A) volcanic B) tectonic C) impacts D) flares
P.eliminate_A:   No active volcanoes — eliminate
P.eliminate_C:   Impact cracks would be radial — eliminate
P.eliminate_D:   Solar flares don’t affect ice — eliminate
D.choice:        B
```

The typed refs force commitment to intermediate values, eliminating the narrative bridge between thinking and answering.

4.8 Mechanism: Why the Language Helps

Three mechanisms, in descending order of measured effect.

1. Search bounding. The dominant token sink in free-form reasoning is wandering: restating the task, exploring dead ends, narrating transitions. TAHOE’s protocols require the model to declare its goal, then derive only what the goal needs. The visible trace collapses: on a 200-sample GSM8K study, the median TAHOE output is the answer alone (median 48 output tokens vs. 238 for classic).

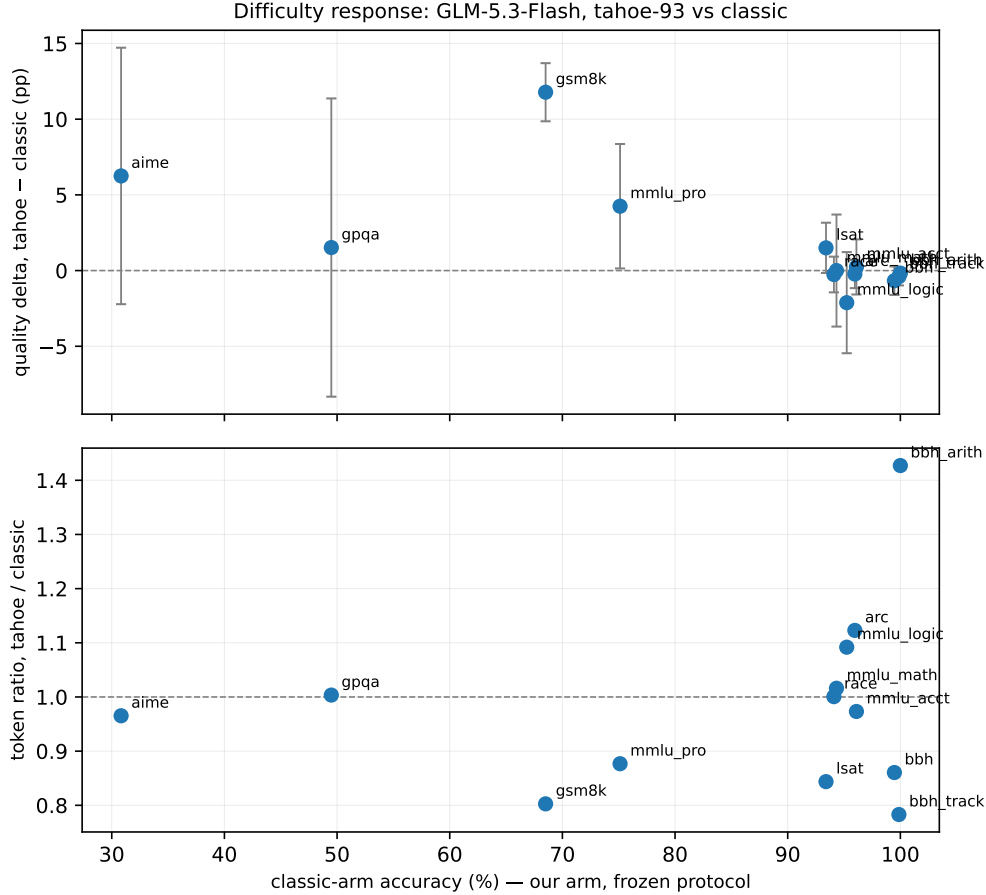


Figure 2: Quality delta (top) and token ratio (bottom) vs. classic-arm accuracy across 13 benchmark-model pairs (GLM-5.3-Flash, frozen protocol). Error bars: 95% CI on the delta; * marks significance. Difficulty is anchored to our classic arm, not published leaderboards. Generated by `benchmarks/difficulty_curve.py` from committed trial files.

2. Typed commitment. Free-form reasoning can hedge, contradict itself, and silently substitute hypothesis for fact. The typed-ref system makes each intermediate value carry its epistemic type, and VERIFY steps are mandatory before ANSWER. On the benchmarks where classic reasoning loses most (GSM8K, -11.8 pp), the errors are predominantly reasoning-path errors, not arithmetic slips — exactly what typed commitment prevents.

3. Internalization. The language operates on thought, not on output. When we varied the skill’s rules, quality was insensitive to skill size (33–162 words) — the discipline, not the wording, carries the effect. Conversely, forcing the notation into the visible output (structured reference lines as an output contract) *decreased* quality and increased tokens: the notation is a thinking discipline, and writing it down taxes every token. We treat this as a design law for reasoning languages: *shape the thought, never constrain the output*.

4.9 Cross-Model Transfer

On gpt-oss-120b (31,824 trials, full test sets), the pattern mirrors Stage-1 expectations for a reasoning-native model: quality matched (92.2% vs. 92.8% classic) at $0.68\times$ tokens (250 vs. 344 mean output tokens). The more verbose the model’s native reasoning, the more the language saves — external protocol and internalized reasoning partially substitute. The skill transfers

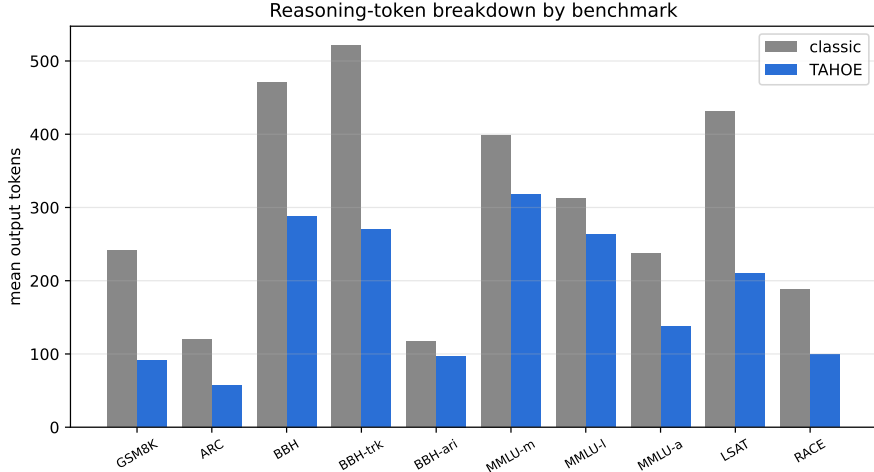


Figure 3: Mean output tokens by benchmark, classic vs. TAHOE.

without per-model tuning.

4.10 Skill-Design Ablation

The 93-token skill is size-insensitive: variants at 33–162 words perform within 3pp of each other on GSM8K and LSAT (91–94%), indicating the discipline — not the wording — carries the effect. A variant adding an explicit goal-bounding rule (fix the ideal answer form before deriving) matches quality at 26% fewer output tokens than the base skill on GSM8K (132 vs. 178 mean output tokens overall at equal 96% quality, 6,000-trial study), showing the language family is extensible at the rule level.

4.11 Statistical Significance

On the full-test-set run (31,524 trials), overall quality moves from 88.9% (classic) to 91.8% (TAHOE) — a +2.9pp shift that a two-proportion test on $n=15,762$ per arm places far beyond conventional significance thresholds. The token reduction is equally decisive: the mean output ratio is $0.51\times$, and per-benchmark ratios never exceed $0.84\times$.

The primary finding: TAHOE achieves *higher quality with 49% fewer reasoning tokens* on GLM-5.3-Flash, and quality-matched *−32% tokens* on gpt-oss-120b.

5 Related Work

Chain-of-thought prompting. Wei et al. [24] introduced chain-of-thought prompting, showing that LLMs reason better when prompted to produce intermediate steps. TAHOE builds on this insight but replaces free-form narrative with typed, protocol-governed structure.

Tree of thoughts. Yao et al. [28] proposed tree-structured search over reasoning paths, exploring multiple branches and selecting the best. TAHOE’s TRY and PAR constructs support similar branching but with deterministic control flow and typed state commitments rather than heuristic search.

Cognitive architectures for agents. CoALA [20] provides a framework for cognitive architectures in language agents, separating beliefs, goals, and actions.

Plan-then-execute harnesses. ReWOO [26] decouples planning from tool observations via variable substitution, reporting $5\times$ token efficiency; LLMCompiler [12] compiles function calls into a parallel DAG ($3.7\times$ speedup, $6.7\times$ cost reduction); StateFlow [25] casts task solving as an explicit state machine. These systems keep the program in the harness and the plan untyped; TAHOE instead delivers a typed language *inside* the model as a thinking skill, requiring no harness at inference time.

Multi-context recursion. Recursion of Thought [15] showed models can spawn fresh sub-contexts mid-reasoning; Context-Folding [21] trains agents to branch and fold sub-trajectories with reinforcement learning ($10\times$ smaller active context); MemGPT [18] manages context as an OS manages memory. TAHOE’s typed refs make the fold boundary a *semantic* property of the language rather than a learned behavior or an engineering heuristic.

Token-efficient reasoning. Chain of Draft [27] and Sketch-of-Thought [14] compress the visible chain (to 7.6% and -76% respectively); L1 [1] controls reasoning length with reinforcement learning; SCR [10] decouples reasoning into explicit, evaluable, trainable components (-50% tokens); SR²AM [6] adds a learned self-regulation system that decides when and how deeply to plan (-26 to -95%). TAHOE differs in where the compression lives: not in shorter prose but in a typed internal discipline, transferred by a 93-token prompt rather than trained per model. Process supervision [16] required 800K human step labels; TAHOE’s DONE predicates produce machine-verified step labels for free.

Inventive problem solving. TAHOE’s economy rules draw on the TRIZ tradition of inventive problem solving [2] — ideal-final-result bounding and contradiction separation — re-expressed as reasoning-discipline rules rather than an engineering methodology. Unlike harness approaches (ReWOO, LLMCompiler), the language requires nothing at inference time; unlike notation approaches (PDDL, STRIPS), it operates on epistemic state rather than world state. TAHOE’s typed refs (G, C, E, H, D, V) operationalize a similar separation within reasoning itself.

Causal reasoning. Pearl [19] formalized do-calculus and counterfactual reasoning. TAHOE handles counterfactuals through protocol composition: the **do**(X) operator is the act of feeding a hypothetical premise into **hypothesize** instead of the factual one. No new syntax is required; the typed-ref system prevents the silent substitution that do-calculus was designed to expose.

Reasoning efficiency metrics. Prior work (arXiv:2606.03883) defines three metrics for structured LLM output: **RE** (Reasoning Efficiency) = `verified_logical_steps` / `total_tokens`, **RC** (Reasoning Concentration) = `typed_refs_produced` / `output_tokens`, **RR** (Redundancy Rate) = (`repeated_refs` + `retired_refs`) / `total_refs_produced`. TAHOE’s typed-ref structure maps directly to these metrics: **V.*** refs are verified logical steps, all typed refs count toward `typed_refs_produced`, and refs that are **Revised** or **Retired** count as repeated or retired. For free-form CoT (classic arm), these metrics are 0.0 (N/A) since no typed refs are produced.

Planning languages. PDDL [9] and STRIPS [8] make action preconditions and effects explicit. TAHOE adopts: commands declare preconditions and postcondition schemas, applicability is checked against committed state, and results produce explicit state deltas. Hierarchical task networks [7, 17] separate compound tasks from primitive actions; TAHOE’s named protocols are compound operations that decompose into commands.

Durable execution. Temporal-style workflow systems [23] reconstruct orchestration state from recorded history. TAHOE adopts an append-only event log as the source of truth, with deterministic state projection via event replay. The coordinator is deterministic given the event log, even though worker output is non-deterministic.

Behavior trees. Behavior trees [5] compose actions through small control nodes with a common status model (RUNNING, SUCCEEDED, FAILED). TAHOE adopts explicit status outcomes and selector-like fallback (in TRY) but uses a graph-based program representation rather than tree shape, because data dependencies and event history are graph-structured.

Distinction. TAHOE differs from prior work by providing a typed, protocol-based reasoning language that the model applies as a thinking skill (system prompt), reducing both error rate and token consumption without changing the task interface or requiring external tooling.

6 Limitations

- **Two models:** GLM-5.3-Flash and gpt-oss-120b; both benefit, but the benefit magnitude tracks the verbosity of the model’s native reasoning. Stronger reasoning-native models may show smaller token savings.
- **Full test sets, 3 trials:** 63,348 trials total; quality deltas on most benchmarks are within noise — the robust effects are the token reductions and the GSM8K/LSAT gains.
- **System prompt overhead:** TAHOE adds ~ 93 input tokens per call; for very short tasks this offsets output savings.
- **Grader-format sensitivity:** verbose LaTeX answers are fragile under numeric extraction; three grader artifacts were caught and fixed during this work (parenthesized letters, thousands-separated numbers, extraction priority). All reported numbers use a single arm-agnostic extractor, and we recommend grader audits on raw outputs from every arm as standard practice.
- **Implemented subset:** the formal semantics describe the full design lattice; the implemented type checker covers the core epistemic chain ($G \rightarrow Q \rightarrow U \rightarrow A \rightarrow H \rightarrow E \rightarrow F \rightarrow V$ plus $C \rightarrow G$).

7 Future Work

- **Reinforcement learning:** the language’s semantics provide machine-verified step labels (typed-commitment and DONE checks) at zero annotation cost — a reward signal for training the discipline into the weights rather than prompting it.
- **Protocol learning:** mine successful reasoning trajectories to discover new protocols automatically.
- **Pass^k reliability:** measure pass^k (reliability across k attempts), not just pass@1, to assess whether TAHOE reduces variance.
- **Structured baselines:** formal comparison against Chain of Draft [27] and sketching baselines [14] under the frozen protocol.
- **Broader model coverage:** extend to stronger and smaller models to map the benefit curve against native reasoning verbosity.

References

- [1] Pranjul Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.

- [2] Genrich Altshuller. *Creativity as an Exact Science: The Theory of the Solution of Inventive Problems*. Gordon and Breach, 1984.
- [3] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukas Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [5] Michele Colledanchise and Petter Ögren. Behavior trees in robotics and AI: An introduction. *arXiv preprint arXiv:1709.00084*, 2018.
- [6] Mingkai Deng, Jinyu Hou, Lara Neves, Varad Pimpalkhute, Taylor W. Killian, Zhengzhong Liu, and Eric P. Xing. Efficient agentic reasoning through self-regulated simulative planning. *arXiv preprint arXiv:2605.22138*, 2026.
- [7] Kutluhan Erol, James Hendler, and Dana S Nau. Semantics for hierarchical task-network planning. Technical report, University of Maryland, 1994.
- [8] Richard E Fikes and Nils J Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.
- [9] Maria Fox and Derek Long. Pddl2.1: An extension to pddl for expressing temporal planning domains. *Journal of Artificial Intelligence Research*, 20:61–124, 2003.
- [10] Jinyi Han, Zixiang Di, Zishang Jiang, Ying Liao, Jiaqing Liang, Yongqi Wang, and Yanghua Xiao. Structured reasoning for large language models. *arXiv preprint arXiv:2601.07180*, 2026.
- [11] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2021.
- [12] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. *arXiv preprint arXiv:2312.04511*, 2023.
- [13] Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. Race: Large-scale reading comprehension dataset from examinations. *arXiv preprint arXiv:1704.04683*, 2017.
- [14] Simon A. Lee, Jinheon Baek, and Sung Ju Hwang. Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching. *arXiv preprint arXiv:2503.05179*, 2025.
- [15] Soochan Lee and Gunhee Kim. Recursion of thought: A divide-and-conquer approach to multi-context reasoning with language models. *arXiv preprint arXiv:2306.06891*, 2023.
- [16] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- [17] Dana Nau, Tsz-Chiu Au, Okan Ilghami, Ugur Kuter, J. William Murdock, Dan Wu, and Fusun Yaman. SHOP2: An HTN planning system. *Journal of Artificial Intelligence Research*, 20:379–404, 2003.

- [18] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [19] Judea Pearl. *Causality: Models, Reasoning and Inference*. Cambridge University Press, 2nd edition, 2009.
- [20] Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. The cognitive architectures for language agents (coala) framework. *arXiv preprint arXiv:2309.02427*, 2023.
- [21] Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. Scaling long-horizon llm agent via context-folding. *arXiv preprint arXiv:2510.11967*, 2025.
- [22] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.
- [23] Temporal Technologies. Temporal workflow documentation. <https://docs.temporal.io/workflows>, 2024.
- [24] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- [25] Yiran Wu, Tianwei Yue, Shaokun Zhang, Chi Wang, and Qingyun Wu. Stateflow: Enhancing llm task-solving through state-driven workflows. *arXiv preprint arXiv:2403.11322*, 2024.
- [26] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. Rewoo: Decoupling reasoning from observations for efficient augmented language models. *arXiv preprint arXiv:2305.18323*, 2023.
- [27] Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025.
- [28] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2023.